

Rapport de sécurité

La traçabilité agricole, de la parcelle au marché.

Version du 2026-10-24

Sécurité — rapport d'analyse, correctifs et risques restants

Analyse réalisée sur l'ensemble de la pile : API (NestJS/PostgreSQL), portail web, application mobile Android, application de bureau Windows, mises à jour et chaîne de livraison. Chaque correctif est couvert par un test automatique (`npm run test:e2e:securite` : 43 contrôles de sécurité + 23 de double authentification, chiffrement et verrouillage, exécuté en mode production en intégration continue ; contrôles de signature dans `e2e-app-update.js` et `desktop/test/updater.js`).

Ce rapport n'est pas un audit indépendant. Il décrit ce que nous avons trouvé, corrigé et testé nous-mêmes. Un test d'intrusion par un tiers reste indispensable avant l'exploitation réelle.

1. Failles trouvées et corrigées

#	Faille	Gravité	Correctif
1	Connexion : réponses et durées différentes selon que le compte existe, est suspendu ou verrouillé (énumération de comptes)	Élevée	Même message « Identifiants invalides » et même temps de calcul (empreinte factice argon2id) dans tous les cas ; tentative sur compte verrouillé journalisée
2	Aucune limite de tentatives de connexion par adresse (force brute, bourrage d'identifiants)	Élevée	10 connexions/min/adresse (`LOGIN_THROTTLE_LIMIT`), 30 renouvellements/min ; verrouillage du compte après 5 échecs / 15 min
3	Mots de passe faibles acceptés	Élevée	Politique : ≥ 10 caractères, ni courant, ni suite évidente, ni l'e-mail, au moins 3 types de caractères (ou ≥ 16) ; appliquée à la création, au changement, à l'amorçage
4	Corps de requête non bornés sur les routes anonymes (déni de service mémoire)	Élevée	2 Mo par défaut ; 10/40 Mo uniquement pour synchronisation, réplication, géodonnées ; 413 au-delà
5	Documentation Swagger exposée en production	Moyenne	Désactivée en production (`ENABLE_API_DOCS=true` pour la réactiver)
6	Réponses d'API susceptibles d'être mises en cache (poste partagé, proxy)	Moyenne	`Cache-Control: no-store` sur `/v1`
7	Exception « mot de passe à changer » trop large (expression non ancrée)	Moyenne	Expression ancrée, seules 4 routes autorisées
8	Algorithme JWT non épinglé	Moyenne	HS256 et émetteur imposés ; jetons `alg: none` ou falsifiés refusés (testé)
9	Dépendances vulnérables (multer, nodemailer, selfsigned/node-forge, Express)	Élevée	NestJS 10→11, Express 5, nodemailer 10, selfsigned 5. `npm audit --omit=dev` : 4 avis modérés restants (js-yaml via Swagger, uuid via exceljs), non exploitables dans cet usage

#	Faille	Gravité	Correctif
10	Mises à jour du poste Windows vérifiées seulement par empreinte fournie par le même serveur (serveur compromis = code arbitraire sur tous les postes)	Critique	Signature Ed25519 hors ligne : le poste refuse tout installateur non signé ou signé pour une autre version/empreinte (pas de rejeu). **Active dès que la clé publique est livrée dans l'application (voir §3)**
11	Application de bureau : ouverture de protocoles arbitraires, navigation hors origine, permissions web	Moyenne	Seuls http/https s'ouvrent à l'extérieur ; navigation externe bloquée ; permissions refusées
12	Android : sauvegarde cloud des données locales, trafic en clair possible, version livrée signée avec la clé de débogage	Élevée	`allowBackup=false`, `usesCleartextTraffic=false`, signature de production par secrets CI

| 13 | Pas de second facteur : un mot de passe volé suffisait | Élevée | Double authentification TOTP (RFC 6238) sur portail et mobile, 8 codes de secours à usage unique, code non rejouable, secret chiffré AES-256-GCM, réinitialisation par un administrateur ; `REQUIRE_2FA=true` l'impose aux droits sensibles et aux administrateurs de plateforme (testé) | | 14 | Verrouillage de compte exploitable pour bloquer un utilisateur depuis n'importe où | Moyenne | Verrouillage par couple (compte, adresse) : l'attaquant ne bloque que sa propre adresse ; verrouillage global du compte seulement après 50 échecs (attaque répartie) | | 15 | Documents lisibles sur le disque / dans les sauvegardes | Moyenne | Chiffrement AES-256-GCM des documents au repos (`DOCUMENTS_KEY`), restitution vérifiée par empreinte ; sauvegardes chiffrables (`BACKUP_PASSPHRASE_FILE`), restauration testée | | 16 | Aucune analyse des fichiers déposés | Moyenne | Analyse antivirus ClamAV optionnelle (`CLAMAV_HOST`), refus en cas d'analyseur injoignable si `CLAMAV_REQUIRED=true` | | 17 | Serveur local vierge initialisable par n'importe qui sur le réseau | Moyenne | Initialisation acceptée seulement depuis le poste serveur, ou avec `X-Bootstrap-Token` (testé sur une vraie interface réseau) | | 18 | Secrets uniquement en variables d'environnement ; clé JWT faible acceptée | Moyenne | Secrets par fichier (`*_FILE`), refus de démarrer en production si `JWT_SECRET` < 32 caractères ou prévisible ; analyse CodeQL, `npm audit` (échec si élevé) et recherche de secrets en intégration continue, mises à jour Dependabot |

Vérifié sans faille : noms de tables dynamiques tous en liste blanche ; injections SQL testées (aucun effet) ; isolation entre coopératives (sécurité par ligne + tests croisés) ; contrôle du type réel des documents et empreinte SHA-256 ; réplication limitée à l'organisation du site ; élévation de privilèges par champs superflus (rejetée).

2. Mesures en place (rappel)

Le mobile conserve ses données dans une base chiffrée (SQLCipher) et les purge quand le serveur révoque l'accès ; les jetons sont dans le stockage sécurisé du téléphone. JWT court + jeton de renouvellement tournant, argon2id, permissions contrôlées côté serveur, sécurité par ligne PostgreSQL avec rôle applicatif non propriétaire, audit, écritures financières non modifiables, idempotence, validation stricte (liste blanche, champs inconnus refusés), helmet, métriques protégées par jeton, CORS restrictif, erreurs sans trace interne.

3. Actions à faire par vous (non automatisables)

1. **Clés de signature des mises à jour** : `node desktop/scripts/sign-release.js keygen <dossier-hors-dépôt>` ; copier la clé publique dans `desktop/src/release-public-key.pem`, reconstruire l'application ; garder la clé privée hors du dépôt (coffre). À chaque version : `node desktop/scripts/sign-release.js sign <installateur.exe> <version> <clé-privée.pem>` puis coller la signature dans l'écran Synchronisation au dépôt. Tant que la clé publique n'est pas livrée, la vérification n'est pas imposée.
2. **Android** : créer un keystore de production (le conserver précieusement) et définir les secrets GitHub `ANDROID_KEYSTORE_BASE64`, `ANDROID_KEYSTORE_PASSWORD`, `ANDROID_KEY_ALIAS`, `ANDROID_KEY_PASSWORD`.
3. **Variables de production** (détail dans `DEPLOIEMENT.md`) : `JWT_SECRET`, `SECRETS_KEY`, `DOCUMENTS_KEY` (trois valeurs aléatoires distinctes, sauvegardées **hors du serveur**), `REQUIRE_2FA=true`, `METRICS_TOKEN`, `TRUST_PROXY` exact, HTTPS en amont ou `TLS_DIR`. Le fichier `docker-compose.prod.yml` refuse de démarrer sans les trois clés.
4. **Activer la double authentification** de chaque administrateur, comptable et caissier dès le déploiement (Mon compte) et conserver les codes de secours.
5. **Serveur local vierge** : l'initialiser depuis le poste serveur (ou définir `BOOTSTRAP_TOKEN`).
6. **Chiffrement du disque et de la base** : à activer chez l'hébergeur / au niveau du système (LUKS, volumes chiffrés) ; sauvegardes avec `BACKUP_PASSPHRASE_FILE`, copiées hors site.
7. **Antivirus** : déployer un conteneur ClamAV et définir `CLAMAV_HOST` (avec `CLAMAV_REQUIRED=true` si possible).
8. **GitHub** : activer le suivi des alertes de sécurité et Dependabot sur le dépôt ; traiter les alertes CodeQL.
9. **Test d'intrusion indépendant** avant l'ouverture à de vrais producteurs : à commander auprès d'un prestataire (nous ne pouvons pas le réaliser nous-mêmes). Fournir `SECURITE.md` et les scripts `e2e-securite.js` / `e2e-2fa.js` comme point de départ.
10. **Données personnelles** : faire valider `DONNEES_PERSONNELLES.md` par un juriste.

4. Risques restants (honnêtement)

- **Aucun test d'intrusion indépendant**, ni test sur appareils Android ou Windows réels : seul un tiers peut les faire.
- **Signature des mises à jour inactive** tant que la clé publique n'est pas livrée (action 1).
- **Chiffrement au repos de la base** : à la charge de l'hébergeur (action 6) ; les documents, eux, sont chiffrés par l'application une fois `DOCUMENTS_KEY` définie (les anciens fichiers restent en clair tant qu'ils ne sont pas redéposés).
- **Perte de `DOCUMENTS_KEY` ou `SECRETS_KEY`** : documents illisibles / 2FA à réinitialiser. Sauvegardez-les à part.
- **`/v1/mode` public** révèle le mode, la version de l'application et celle du schéma (nécessaires au jumelage des postes) : information de faible valeur, acceptée.
- **Authentification à deux facteurs facultative** si `REQUIRE_2FA` n'est pas activé ; les comptes de terrain (délégués, coachs) n'y sont pas soumis par la règle.

- **Verrouillage global** après 50 échecs toutes adresses confondues : un attaquant très distribué peut encore gêner un compte (déni de service) ; la limite par adresse et la double authentification protègent contre l'intrusion.
- **Antivirus facultatif** (action 7) ; formats limités à PDF, JPEG, PNG.
- **Secrets de la CI** (keystore) et clés privées : leur protection dépend de votre organisation.
- **Conformité légale** : à valider par un juriste (`DONNEES_PERSONNELLES.md`).

5. Procédure de révocation d'un accès (CDC §27.7)

1. `POST /users/:id/suspend` (permission `users:suspend`, motif obligatoire) : accès refusé dès la requête suivante.
2. Appareil perdu : `POST /users/:id/revoke-sessions` ; la synchronisation se bloque.
3. Changement de droits : `POST /users/:id/role`, effet immédiat.
4. Tout est tracé (`user_access_events`, audit) ; un administrateur actif doit toujours rester.

6. En cas d'incident

Suspendre les comptes concernés, révoquer les sessions, changer `JWT_SECRET` (déconnecte tout le monde ; la 2FA et les documents ne sont pas touchés, ils dépendent de `SECRETS_KEY` et `DOCUMENTS_KEY`), consulter l'audit et les journaux (`auth.login_while_locked`, échecs répétés), restaurer depuis la sauvegarde si nécessaire (`scripts/restore-test.sh`).